Machine learning

Probabilistic graphical models

Thomas D. Nielsen

Probabilistic modeling: Recap from last time

Probabilistic graphical models (PGMs): syntax

Let $X_1, \ldots, X_n$ be a collection of random variables. A PGM over $X_1, \ldots, X_n$ consists of

- a directed acyclic graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$ whose nodes $\mathcal{V}$ are the variables $X_1, \ldots, X_n$
- a set of local conditional distributions, $\mathcal{P} = \{p(X_i \mid \mathrm{pa}(X_i)), X_i \in \mathcal{V}\}$, where $\mathrm{pa}(X_i)$ are the parents of X_i in $\mathcal{G}$ as defined by the edges $\mathcal{E}$.

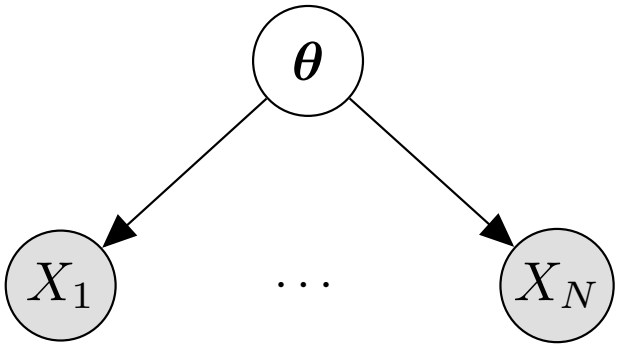
Probabilistic graphical models: semantics

A PGM $\mathcal{N}$ with nodes $X_1, \ldots, X_n$ defines a joint distribution

$$p_{\mathcal{N}}(X_1, \ldots, X_n) = \prod_{i=1}^{n} p(X_i \mid \mathrm{pa}(X_i))$$

Plate notation

Two representations of a graphical model for N independent thumbtack tosses:



Unfolded notation

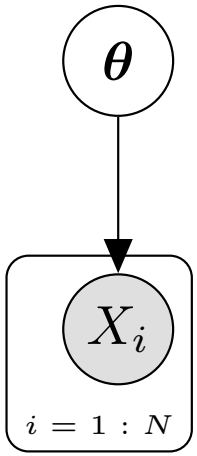


Plate notation

$\theta \sim \text{Beta}(2, 5)$

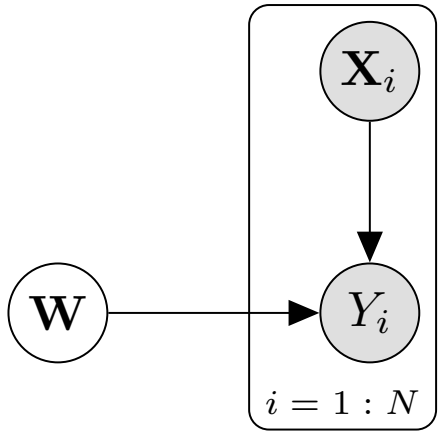
$\theta \sim p(X_i = 1 \mid \theta) = \theta$

The idea is to avoid repeated substructures

Probabilistic modeling

- Specify prior beliefs
- Provide a probabilistic description/explanation of how the data is generated

Example



(Bayesian) linear regression model

- Only structure specified (using plate notation)

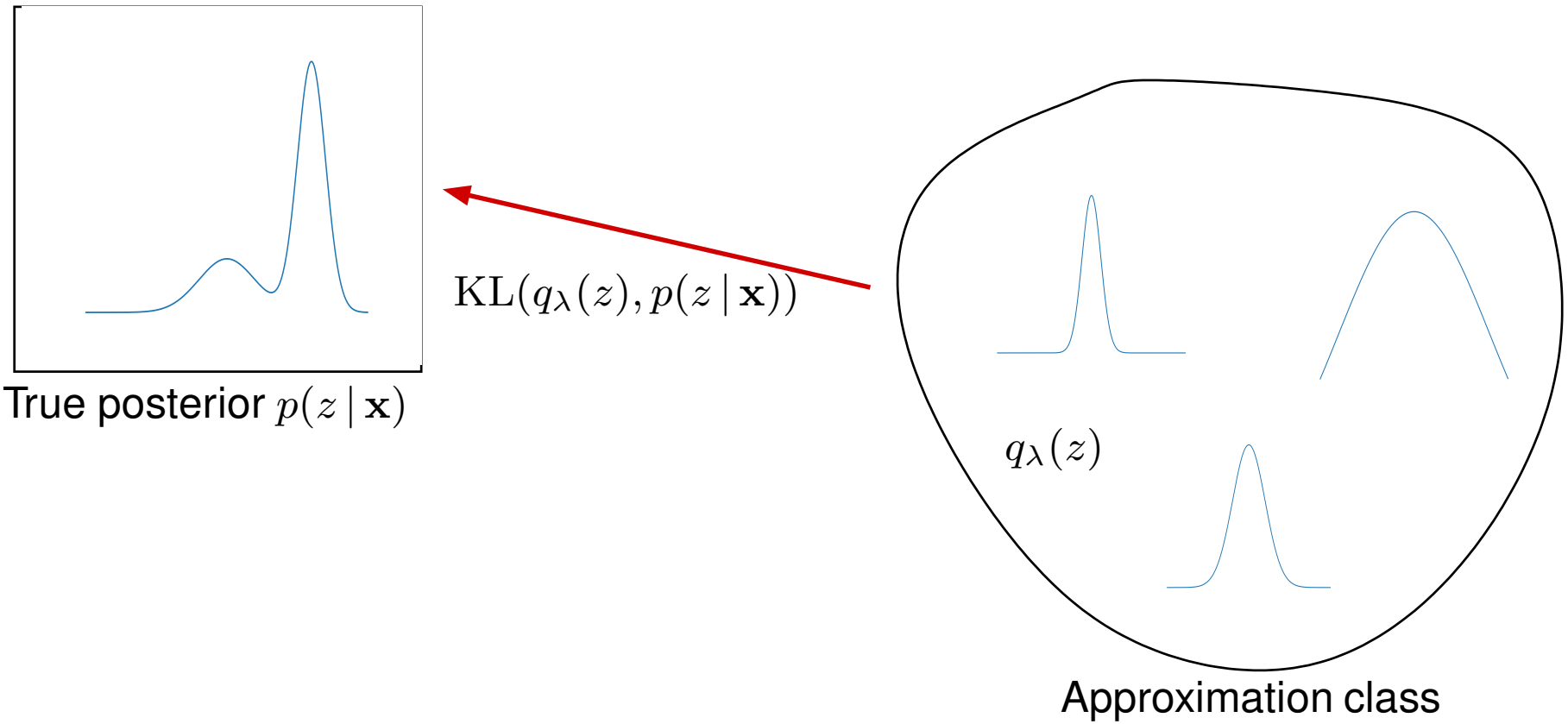
Bayes' rule Bayes' rule illustrates some of the inference problems we face:

$$\underbrace{p(\mathbf{W} \mid \mathbf{y}, \mathbf{x})}_{\text{Posterior}} = \frac{\overbrace{p(y \mid \mathbf{x}, \mathbf{W})}^{\text{Likelihood}} \overbrace{p(\mathbf{W})}^{\text{Prior}}}{\underbrace{\int p(y, \mathbf{x}, \mathbf{W}) \mathrm{d}\mathbf{W}}_{\text{Marginal likelihood/model evidence}}}$$

Approximate inference: Variational inference

What is variational inference?

We will approximate the true posterior distribution $p(z \mid \mathbf{x})$ with a variational distribution belonging to a tractable family of distributions.



True posterior $p(z \mid \mathbf{x})$

Approximation class

Task: Fit the variational parameters λ so that the 'distance' $\mathrm{KL}(q_\lambda(z), p(z \mid \mathbf{x}))$ is minimized:

$$\hat{q}(z) = \arg\min_{\lambda} \mathrm{KL}(q_\lambda(z), p(z \mid \mathbf{x}))$$

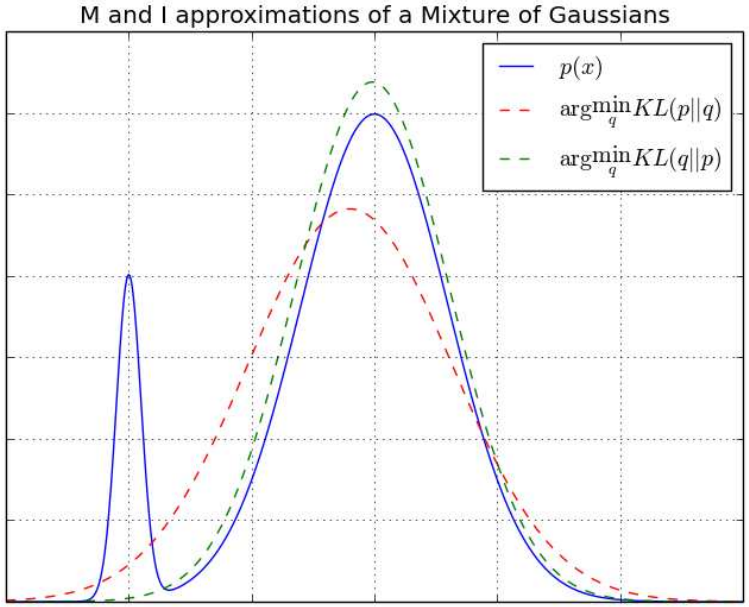
The KL-divergence:

$$\mathrm{KL}\left(f||g\right) = \int_{\mathbf{z}} f(\mathbf{z}) \log\left(\frac{f(\mathbf{z})}{g(\mathbf{z})}\right) \mathrm{d}\mathbf{z} = \mathbb{E}_f\left[\log\left(\frac{f(\mathbf{z})}{g(\mathbf{z})}\right)\right].$$

is *not* symmetric ($\mathrm{KL}\left(f||g\right) \neq \mathrm{KL}\left(g||f\right)$ in general). In variational inference we focus on $\mathrm{KL}\left(q||p\right)$:

$$\hat{q}(z) = \arg\min_{\lambda} \mathrm{KL}(q_\lambda(z), p(z \mid \mathbf{x}))$$

Example



The complex distribution p is approximated by a simpler variational distribution q.

ELBO: Evidence Lower-BOund

We can rearrange the KL divergence as follows:

$$\mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right) \quad = \quad \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})}\right]$$

ELBO: Evidence Lower-BOund

We can rearrange the KL divergence as follows:

$$\begin{aligned}\mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right) &= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})}\right] \\ &= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})\cdot p(\mathbf{x})}{p(\mathbf{z}\,|\,\mathbf{x})\cdot p(\mathbf{x})}\right]\end{aligned}$$

ELBO: Evidence Lower-BOund

We can rearrange the KL divergence as follows:

$$\begin{aligned}
\mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right) &= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})}\right] \\
&= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})\cdot p(\mathbf{x})}{p(\mathbf{z}\,|\,\mathbf{x})\cdot p(\mathbf{x})}\right] = \log p(\mathbf{x}) + \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})\cdot p(\mathbf{x})}\right]
\end{aligned}$$

ELBO: Evidence Lower-BOund

We can rearrange the KL divergence as follows:

$$\begin{aligned}
\mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right) &= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})}\right] \\
&= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})\cdot p(\mathbf{x})}{p(\mathbf{z}\,|\,\mathbf{x})\cdot p(\mathbf{x})}\right] = \log p(\mathbf{x}) + \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})\cdot p(\mathbf{x})}\right] \\
&= \log p(\mathbf{x}) - -\mathbb{E}_q\left[\log \tfrac{q(\mathbf{z})}{p(\mathbf{z},\mathbf{x})}\right] = \log p(\mathbf{x}) - \mathcal{L}\left(q\right)
\end{aligned}$$

where $\mathcal{L}\left(q\right) = -\mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z},\mathbf{x})}\right]$ is the so-called Evidence Lower Bound (ELBO)

ELBO: Evidence Lower-BOund

We can rearrange the KL divergence as follows:

$$\begin{aligned}
\mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right) &= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})}\right] \\
&= \mathbb{E}_q\left[\log \frac{q(\mathbf{z})\cdot p(\mathbf{x})}{p(\mathbf{z}\,|\,\mathbf{x})\cdot p(\mathbf{x})}\right] = \log p(\mathbf{x}) + \mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}\,|\,\mathbf{x})\cdot p(\mathbf{x})}\right] \\
&= \log p(\mathbf{x}) - -\mathbb{E}_q\left[\log \tfrac{q(\mathbf{z})}{p(\mathbf{z},\mathbf{x})}\right] = \log p(\mathbf{x}) - \mathcal{L}\left(q\right)
\end{aligned}$$

where $\mathcal{L}\left(q\right) = -\mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z},\mathbf{x})}\right]$ is the so-called Evidence Lower Bound (ELBO)

VI focuses on the ELBO:

$$\log p(\mathbf{x}) = \mathcal{L}\left(q\right) + \mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right)$$

Since $\log p(\mathbf{x})$ is constant wrt. q and $\mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right) \geq 0$ it follows:

- We can minimize $\mathrm{KL}\left(q(\mathbf{z})||p(\mathbf{z}\,|\,\mathbf{x})\right)$ by maximizing $\mathcal{L}\left(q\right)$
- This is **computationally simpler** because it uses $p(\mathbf{z},\mathbf{x})$ instead of $p(\mathbf{z}\,|\,\mathbf{x})$.
- $\mathcal{L}\left(q\right)$ is a *lower bound* of the marginal data log likelihood $\log p(\mathbf{x})$.

$\rightsquigarrow$ During inference, we will look for $\hat{q}(\mathbf{z}) = \arg\max_{q\in\mathcal{Q}} \mathcal{L}\left(q\right)$.

Assumptions about the variational distribution

The mean field assumption

We will often use the mean field assumption, which states that $\mathcal{Q}$ consists of all distributions that *factorizes* according to the equation

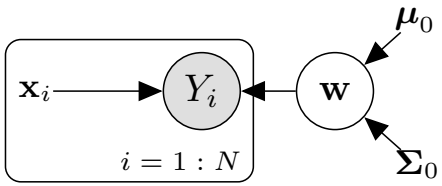
$$q(\mathbf{z}) = \prod_i q_i(z_i)$$

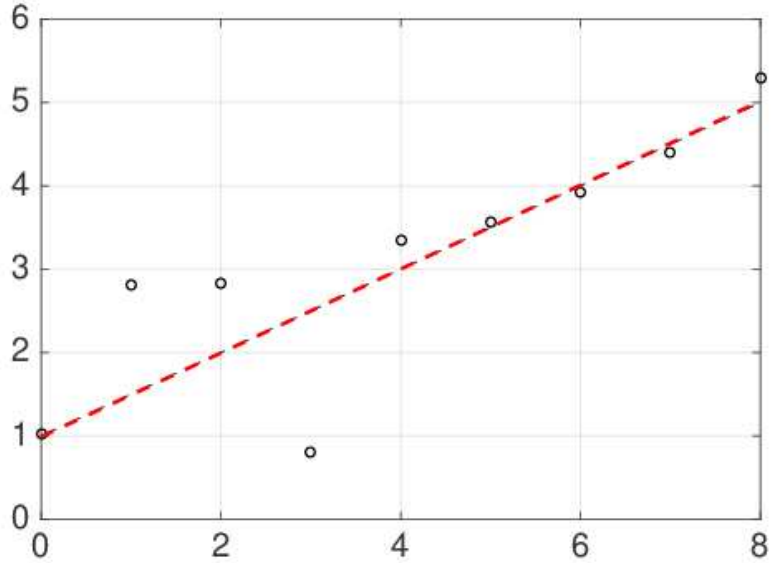
Note! This may seem like a very restricted set. However, we can choose any $q(\mathbf{z}) \in \mathcal{Q}$, and this is how the magic ($\sim$ "absorbing information from $\mathbf{x}$") happens.

VB-MF example – “sanity check”

Simple example:

- Regression model:
  $Y_i \mid \{\mathbf{w}, \mathbf{x}_i\} = \mathbf{w}^\mathsf{T}\mathbf{x}_i + \epsilon_i$.
  - For notational convenience we write $\mathbf{x}_i$ for $[1, x_i]^\mathsf{T}$, x_i is a scaler.
- $\epsilon \sim N(0, 1/\gamma)$; γ known.
- $\mathbf{w} \sim \mathcal{N}\left(\boldsymbol{\mu}_0 = \mathbf{0}, \boldsymbol{\Sigma}_0 = \mathbf{I}_{2\times 2}\right)$.

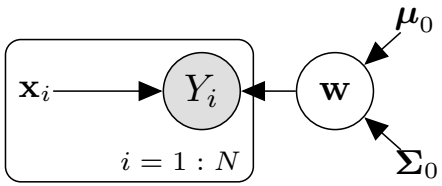


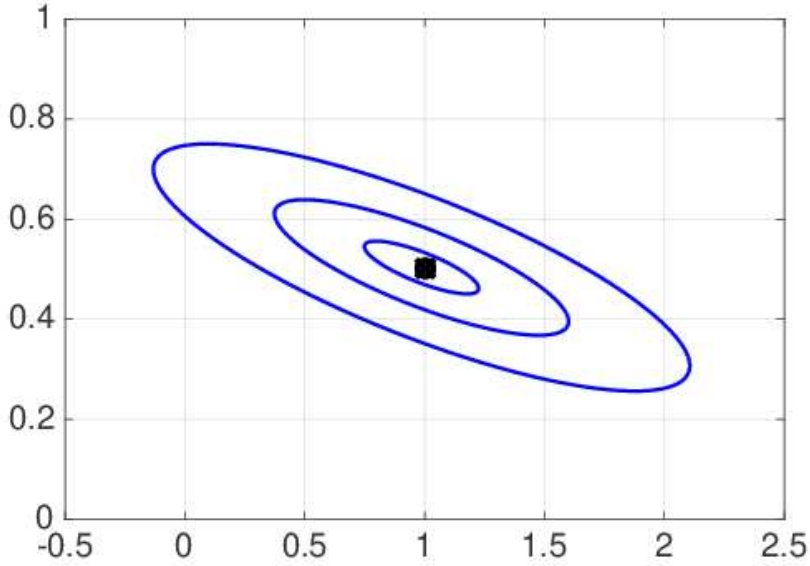


VB-MF example – “sanity check”

Simple example:

- Regression model:
  $Y_i \mid \{\mathbf{w}, \mathbf{x}_i\} = \mathbf{w}^\mathsf{T}\mathbf{x}_i + \epsilon_i$.
  - For notational convenience we write $\mathbf{x}_i$ for $[1, x_i]^\mathsf{T}$, x_i is a scaler.
- $\epsilon \sim N(0, 1/\gamma)$; γ known.
- $\mathbf{w} \sim \mathcal{N}\left(\boldsymbol{\mu}_0 = \mathbf{0}, \boldsymbol{\Sigma}_0 = \mathbf{I}_{2\times 2}\right)$.



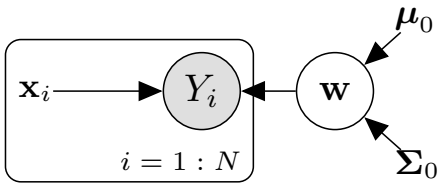


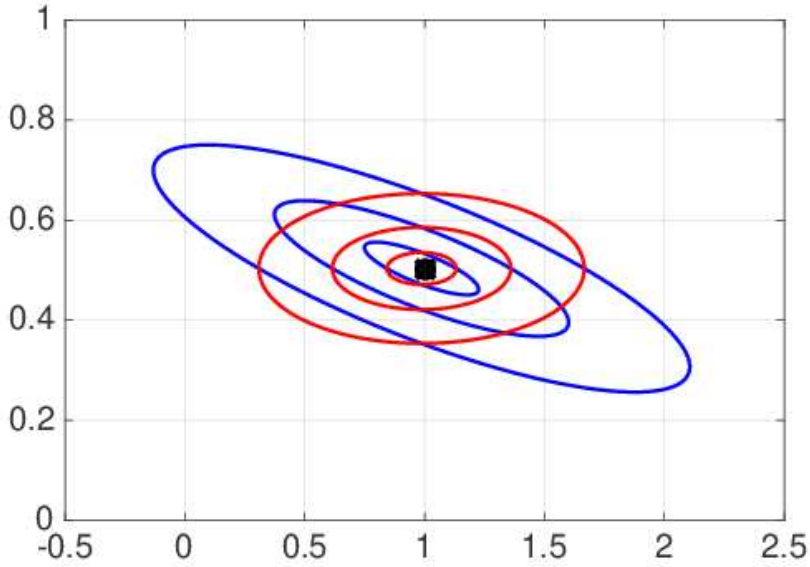
- The exact Bayesian solution

VB-MF example – “sanity check”

Simple example:

- Regression model:
  $Y_i \mid \{\mathbf{w}, \mathbf{x}_i\} = \mathbf{w}^\mathsf{T}\mathbf{x}_i + \epsilon_i$.
  - For notational convenience we write $\mathbf{x}_i$ for $[1, x_i]^\mathsf{T}$, x_i is a scaler.
- $\epsilon \sim N(0, 1/\gamma)$; γ known.
- $\mathbf{w} \sim \mathcal{N}(\boldsymbol{\mu}_0 = \mathbf{0}, \boldsymbol{\Sigma}_0 = \mathbf{I}_{2\times 2})$.



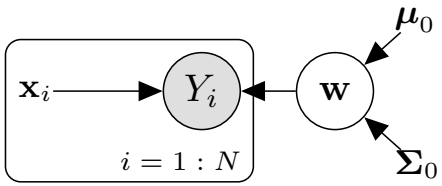


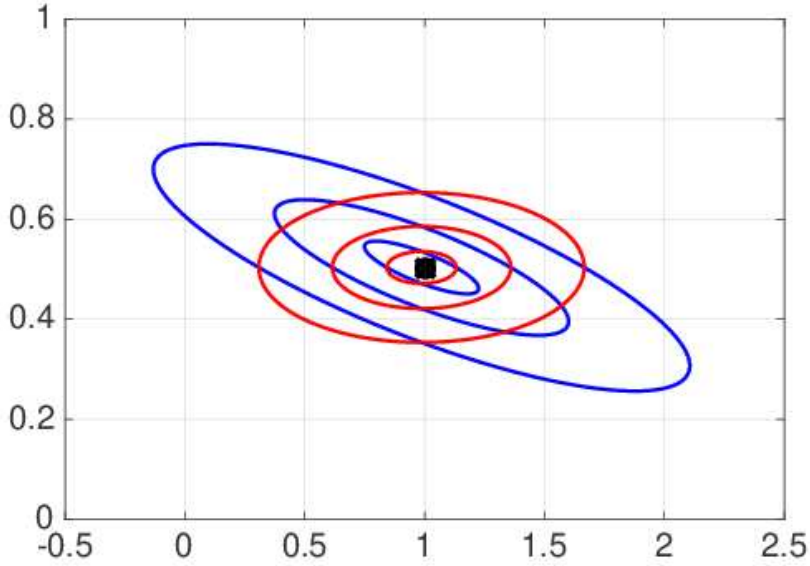
- The exact Bayesian solution
- The variational solution w/ mean field.

VB-MF example – "sanity check"

Simple example:

- Regression model:
  $Y_i \mid \{\mathbf{w}, \mathbf{x}_i\} = \mathbf{w}^\mathsf{T}\mathbf{x}_i + \epsilon_i$.
  - For notational convenience we write $\mathbf{x}_i$ for $[1, x_i]^\mathsf{T}$, x_i is a scaler.
- $\epsilon \sim N(0, 1/\gamma)$; γ known.
- $\mathbf{w} \sim \mathcal{N}\left(\boldsymbol{\mu}_0 = \mathbf{0}, \boldsymbol{\Sigma}_0 = \mathbf{I}_{2\times 2}\right)$.





- The exact Bayesian solution
- The variational solution w/ mean field.

Observations

- The VB-MF solution approximates the mean of the true posterior well.
- VB-MF totally disregards correlation between W_1 and W_2.
- VB-MF under-estimates the uncertainty of the true posterior, as shown by the 10%, 50% and 90% credibility intervals.

Interpreting the ELBO

The ELBO (rewritten)

$$\mathcal{L}(q) = -\mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z},\mathbf{x})}\right] = \mathbb{E}_q\left[\log \frac{p(\mathbf{z},\mathbf{x})}{q(\mathbf{z})}\right]$$

$$= \underbrace{\mathbb{E}_q[\log p(\mathbf{x}\,|\,\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{KL(q(\mathbf{z}),p(\mathbf{z}))}_{\text{Penalty term}}$$

Interpreting the bound

- Reconstruction term: The expected log-likelihood measuring how well samples from $q(\mathbf{z})$ explains the data $\mathbf{x}$.
- Penalty term: Penalizes large deviations between $q(\mathbf{z})$ and the prior $p(\mathbf{z})$. Can be interpreted as a form of regularization.

Variational (Bayes) inference for learning

Bayesian learning using variational inference:

- We posit a model $p(\mathbf{y} \mid \boldsymbol{\theta})$, and want to learn $\boldsymbol{\theta}$ from a data-set $\mathcal{D} = \{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$.
- We cast the problem in a Bayesian setting by including a prior $p(\boldsymbol{\theta})$.
- We seek the posterior $p(\boldsymbol{\theta} \mid \mathcal{D})$, which is approximated by

$$\hat{q}(\boldsymbol{\theta}) = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\left(q(\boldsymbol{\theta}) || p(\boldsymbol{\theta} \mid \mathcal{D})\right).$$

- This is identical to maximizing $\mathcal{L}(q)$ – a lower-bound of $p(\mathcal{D})$.

Summary

- Approximate inference can be seen as *optimization*, where we look for a "simple" distribution $q(\mathbf{z})$ that is close to a "difficult" distribution $p(\mathbf{z} \mid \mathbf{x})$.
- First we define a class $\mathcal{Q}$ of functions that are considered simple. In particular the *mean-field* assumption focus on distributions that *factorize*:

$$q(\mathbf{z}) = \prod_i q_i(z_i \mid \boldsymbol{\lambda}_i).$$

- In VB, parameters for each q_i are chosen to minimize $\mathrm{KL}\left(q||p\right)$.
- Practically we define the Evidence Lower Bound (ELBO),

$$\mathcal{L}\left(q\right) = -\mathbb{E}_q\left[\log \frac{q(\mathbf{z})}{p(\mathbf{z}, \mathbf{x})}\right],$$

and look for $\hat{q}(\mathbf{z}) = \arg\max_{q \in \mathcal{Q}} \mathcal{L}\left(q\right)$.

Black Box Variational Inference

VI inference as optimization

We can minimize (improve the variational approximation)

$$\mathrm{KL}(q_\lambda(z), p(z \mid \mathbf{x}))$$

by maximizing the ELBO

$$\mathcal{L}(q) = \mathbb{E}_q \left[\log \frac{p(\mathbf{z}, \mathbf{x})}{q(\mathbf{z})} \right]$$

VI inference as optimization

We can minimize (improve the variational approximation)

$$\mathrm{KL}(q_\lambda(z), p(z \mid \mathbf{x}))$$

by maximizing the ELBO

$$\mathcal{L}(q) = \mathbb{E}_q \left[\log \frac{p(\mathbf{z}, \mathbf{x})}{q(\mathbf{z})} \right]$$

The mean field assumption

Again, we will use the mean field assumption, which states that $\mathcal{Q}$ consists of all distributions that *factorizes* according to the equation

$$q(\mathbf{z}) = \prod_i q_i(z_i)$$

$\rightsquigarrow$ we can treat the variables independently.

Vanilla black box variational inference

Algorithm: Maximize $\mathcal{L}(q) = \mathbb{E}_{q_\lambda}\left[\log \frac{p_\theta(\mathbf{z},\mathbf{x})}{q_\lambda(\mathbf{z})}\right]$ by gradient ascent

- Initialization:
  - $t \leftarrow 0$;
  - $\hat{\boldsymbol{\lambda}}_0 \leftarrow$ random initialization;
- Repeat until negligible improvement in terms of $\mathcal{L}(q)$:
  - $t \leftarrow t + 1$;
  - $\hat{\boldsymbol{\lambda}}_t \leftarrow \hat{\boldsymbol{\lambda}}_{t-1} + \rho_t \, \nabla_\lambda \mathcal{L}(q)|_{\hat{\boldsymbol{\lambda}}_{t-1}}$;

Note: We use λ to refer to the parameters of the variational distribution $q_\lambda(\mathbf{z})$.

BBVI - calculating the gradient

The algorithm requires that we can find

$$\nabla_\lambda \mathcal{L}(q) = \nabla_\lambda \mathbb{E}_q \left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z})} \right].$$

With a bit of pencil pushing we end up with

$$\nabla_\lambda \mathcal{L}(q) = \mathbb{E}_{q_\lambda} \left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z})} \cdot \nabla_\lambda \log q_\lambda(\mathbf{z}) \right],$$

which turns out to be easier to evaluate.

Calculating the gradient – Things to notice

$$\nabla_\lambda \mathcal{L}(q) = \mathbb{E}_{q_\lambda}\left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})} \cdot \nabla_\lambda \log q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})\right].$$

Calculating the gradient – Things to notice

- We only need access to the un-normalized $p_\theta(\mathbf{z}, \mathbf{x})$ – not $p_\theta(\mathbf{z} \mid \mathbf{x})$.

$$\nabla_\lambda \mathcal{L}(q) = \mathbb{E}_{q_\lambda}\left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})} \cdot \nabla_\lambda \log q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})\right].$$

Calculating the gradient – Things to notice

- We only need access to the un-normalized $p_\theta(\mathbf{z}, \mathbf{x})$ – not $p_\theta(\mathbf{z} \mid \mathbf{x})$.

$$\nabla_\lambda \mathcal{L}(q) = \mathbb{E}_{q_\lambda}\left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})} \cdot \nabla_\lambda \log q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})\right].$$

- $q_\lambda(\mathbf{z})$ factorizes under MF, s.t. we can optimize per variable: $q_{\lambda_i}(z_i)$.

Calculating the gradient – Things to notice

- We only need access to the un-normalized $p_\theta(\mathbf{z}, \mathbf{x})$ – not $p_\theta(\mathbf{z} \mid \mathbf{x})$.

$$\nabla_\lambda \mathcal{L}(q) = \mathbb{E}_{q_\lambda}\left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})} \cdot \nabla_\lambda \log q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})\right].$$

- $q_\lambda(\mathbf{z})$ factorizes under MF, s.t. we can optimize per variable: $q_{\lambda_i}(z_i)$.
- We must calculate $\nabla_\lambda \log q(\mathbf{z} \mid \boldsymbol{\lambda})$, which is also known as the "score function".

Example

If $q_{\boldsymbol{\lambda}}(z)$ follows a normal distribution ($\boldsymbol{\lambda} = (\mu, \sigma)$):

$$\frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(z-\mu)^2}{2\sigma^2}\right),$$

then

$$\nabla_\mu \log q_{\boldsymbol{\lambda}}(z) = \frac{1}{\sigma^2}(z - \mu)$$

Calculating the gradient – Things to notice

- We only need access to the un-normalized $p_\theta(\mathbf{z}, \mathbf{x})$ – not $p_\theta(\mathbf{z} \mid \mathbf{x})$.

$$\nabla_\lambda \mathcal{L}(q) = \mathbb{E}_{q_\lambda}\left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})} \cdot \nabla_\lambda \log q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})\right].$$

- $q_\lambda(\mathbf{z})$ factorizes under MF, s.t. we can optimize per variable: $q_{\lambda_i}(z_i)$.
- We must calculate $\nabla_\lambda \log q(\mathbf{z} \mid \boldsymbol{\lambda})$, which is also known as the "score function".
- The expectation will be approximated using a sample $\{\mathbf{z}_1, \ldots, \mathbf{z}_M\}$ generated from $q(\mathbf{z} \mid \boldsymbol{\lambda})$. Hence we require that we can **sample from** $q_{\lambda_i}(\cdot)$.

Calculating the gradient – Things to notice

- We only need access to the un-normalized $p_\theta(\mathbf{z}, \mathbf{x})$ – not $p_\theta(\mathbf{z} \mid \mathbf{x})$.

$$\nabla_\lambda \mathcal{L}(q) = \mathbb{E}_{q_\lambda}\left[\log \frac{p_\theta(\mathbf{z}, \mathbf{x})}{q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})} \cdot \nabla_\lambda \log q_\lambda(\mathbf{z} \mid \boldsymbol{\lambda})\right].$$

- $q_\lambda(\mathbf{z})$ factorizes under MF, s.t. we can optimize per variable: $q_{\lambda_i}(z_i)$.
- We must calculate $\nabla_\lambda \log q(\mathbf{z} \mid \boldsymbol{\lambda})$, which is also known as the "score function".
- The expectation will be approximated using a sample $\{\mathbf{z}_1, \ldots, \mathbf{z}_M\}$ generated from $q(\mathbf{z} \mid \boldsymbol{\lambda})$. Hence we require that we can **sample from** $q_{\lambda_i}(\cdot)$.

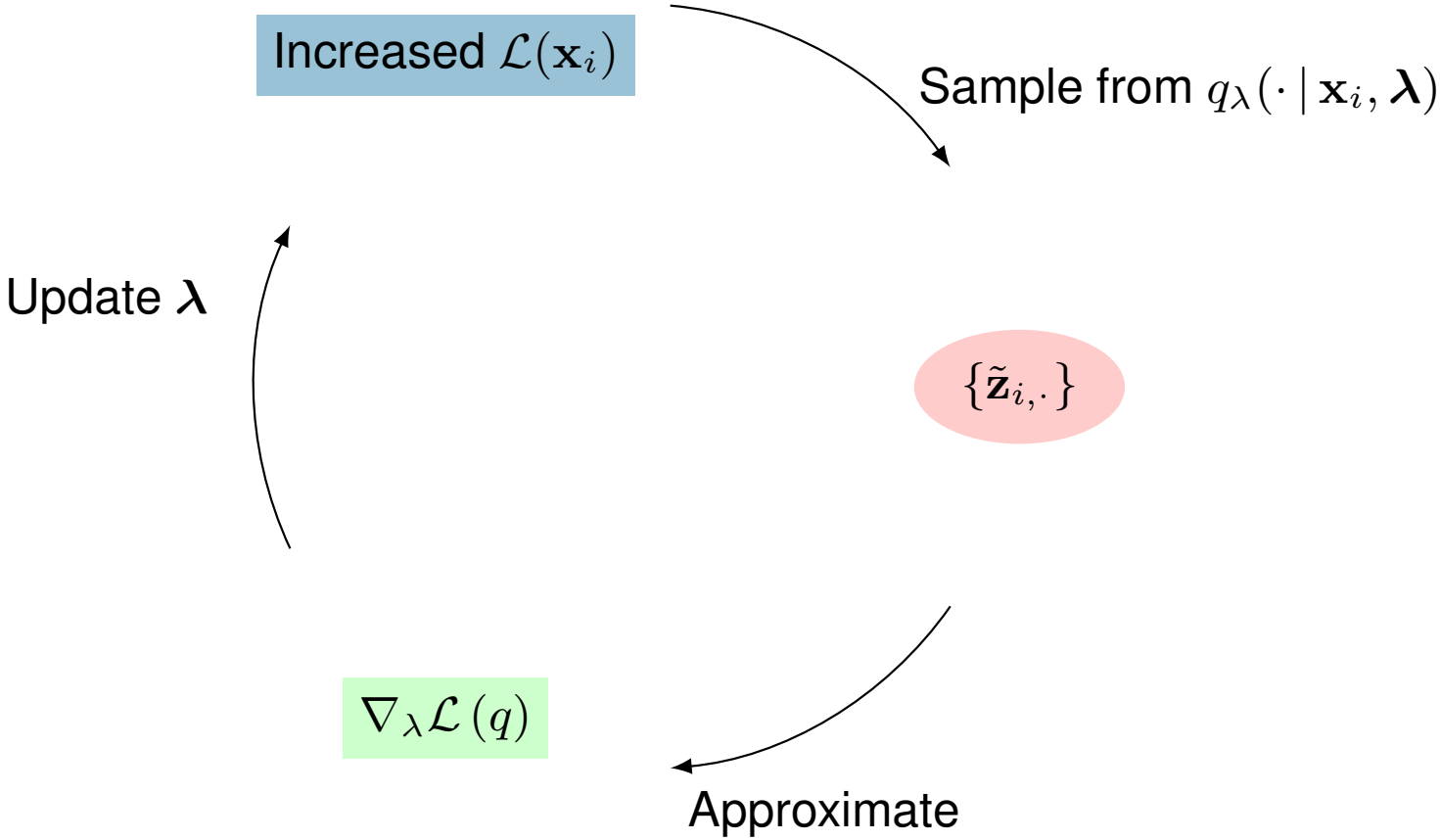
Calculating the gradient – in summary

We have observed the datapoint $\mathbf{x}$, and our current estimate for $\boldsymbol{\lambda}_i$ is $\hat{\boldsymbol{\lambda}}_i$. Then

$$\nabla_{\lambda_i} \mathcal{L}(q)|_{\lambda=\hat{\lambda}_i} \approx \frac{1}{M} \sum_{j=1}^{M} \log \frac{p(z_{i,j}, \mathbf{x})}{q(z_{i,j} \mid \hat{\lambda}_i)} \cdot \nabla_{\lambda_i} \log q_i(z_{i,j} \mid \hat{\lambda}_i).$$

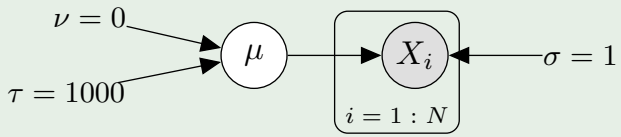
where $\{z_{i,1}, \ldots z_{i,M}\}$ are samples from $q_{\lambda_i}(\cdot \mid \hat{\lambda}_i)$.

ELBO optimization



Example model

We assume a simple generative model:

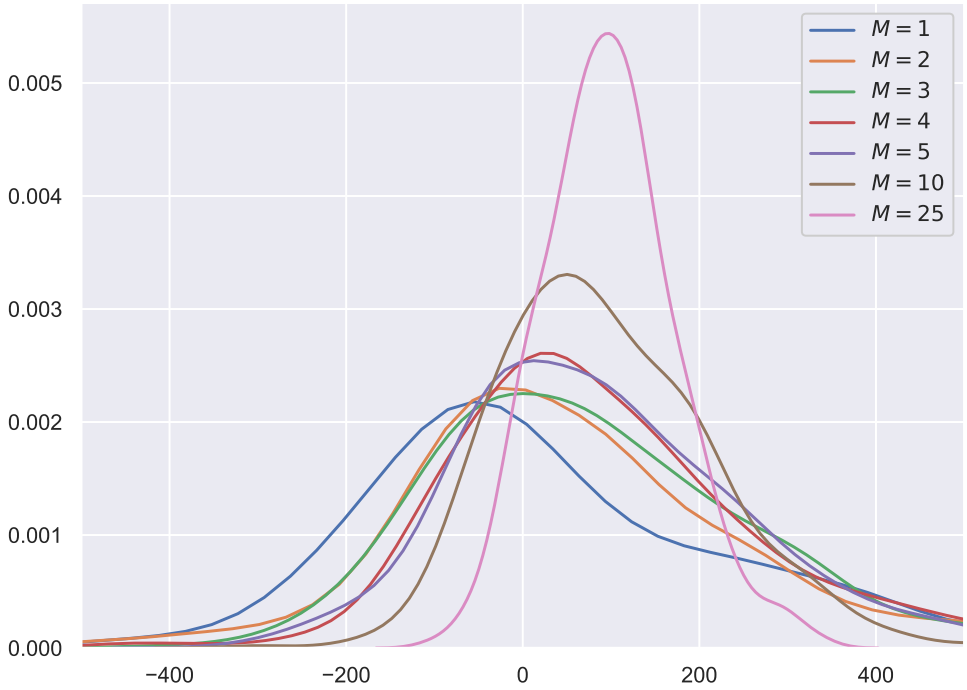


Now, data is generated using $\mu = 10$, we assume a variational model with $q(\mu \mid \lambda) = \mathcal{N}(\lambda, 1)$ and want to maximize the ELBO wrt. the posterior mean for μ in the variational formulation.

student_BBVI.ipynb

BBVI in Python – Evaluating the results

PDF for the gradient calculated at $\lambda = 9$, which is below the optimum ≈ 10.
Several values for M, the sample size used to generate the estimate, are shown.



- Since the gradient estimate is based on a random sample, it is meaningful to evaluate the estimators' "robustness" in terms of a density function.
- We would hope to see robust estimates, also for small M, and in particular high probability for moving in the correct direction (gradient larger than 0).
- This is not the case, which has lead to a major focus on **variance reduction techniques**: while important we will **not cover them here**.

Probabilistic Programming Languages: Another look

Pyro

Pyro (`pyro.ai`) is a Python library for probabilistic modeling, inference, and criticism, integrated with PyTorch.

Modeling:
- Directed graphical models
- Neural networks (via libraries such as `nn.Module`)
- ...

Inference:
- Variational inference – including BBVI
- Monte Carlo – including Gibbs, Hamiltonian Monte Carlo
- ...

Criticism:
- Point-based evaluations
- Posterior predictive checks
- ...

... and there are also many other possibilities

`Tensorflow`, `Edward`, `InferPy`, etc.

Simple example

$$\begin{aligned} \text{temp} &\sim \mathcal{N}(15, 2) \\ \text{sensor} &\sim \mathcal{N}(\text{temp}, 1) \end{aligned}$$

$$p(\text{sensor} = 18, \text{temp})$$

Pyro models

Simple example

$$\begin{aligned} \text{temp} &\sim \mathcal{N}(15, 2) \\ \text{sensor} &\sim \mathcal{N}(\text{temp}, 1) \end{aligned}$$

$$p(\text{sensor} = 18, \text{temp})$$

Pyro models:

- random variables $\Leftrightarrow$ `pyro.sample`
- observations $\Leftrightarrow$ `pyro.sample` with the `obs` argument

Pyro models

Simple example

$$\begin{aligned} \text{temp} &\sim \mathcal{N}(15, 2) \\ \text{sensor} &\sim \mathcal{N}(\text{temp}, 1) \end{aligned}$$

$$p(\text{sensor} = 18, \text{temp})$$

Pyro models:

- random variables $\Leftrightarrow$ `pyro.sample`
- observations $\Leftrightarrow$ `pyro.sample` with the `obs` argument

```
#The observatons
obs = {'sensor': torch.tensor(18.0)}

def model(obs):
    temp = pyro.sample('temp', dist.Normal(15.0, 2.0))
    sensor = pyro.sample('sensor', dist.Normal(temp, 1.0), obs=obs['sensor'])
```

Inference Problem

$$p(\text{temp}|\text{sensor} = 18)$$

Inference Problem

$$p(\text{temp}|\text{sensor} = 18)$$

Variational Solution

$$\min_q \mathrm{KL}\left(q(\text{temp})||p(\text{temp}|\text{sensor} = 18)\right)$$

Pyro guides

Inference Problem

$$p(\text{temp}|\text{sensor} = 18)$$

Variational Solution

$$\min_{q} \mathrm{KL}\left(q(\text{temp})||p(\text{temp}|\text{sensor} = 18)\right)$$

Pyro Guides:

- Defines the q **distributions** in variational settings.

Pyro Guides:

- Guides are **arbitrary stochastic functions**.
- Guides produces samples for those variables of the model which are **not observed**.

Example

```
#The observatons
obs = {'sensor': torch.tensor(18.0)}

def model(obs):
    temp = pyro.sample('temp', dist.Normal(15.0, 2.0))
    sensor = pyro.sample('sensor', dist.Normal(temp, 1.0), obs=obs['sensor'])
```

```
#The guide
def guide(obs):
    a = pyro.param("mean", torch.tensor(0.0))
    b = pyro.param("scale", torch.tensor(1.), constraint=constraints.positive)
    temp = pyro.sample('temp', dist.Normal(a, b))
```

Pyro Guides:

- Guides are **arbitrary stochastic functions**.
- Guides produces samples for those variables of the model which are **not observed**.

Example

```
#The observatons
obs = {'sensor': torch.tensor(18.0)}

def model(obs):
    temp = pyro.sample('temp', dist.Normal(15.0, 2.0))
    sensor = pyro.sample('sensor', dist.Normal(temp, 1.0), obs=obs['sensor'])
```

```
#The guide
def guide(obs):
    a = pyro.param("mean", torch.tensor(0.0))
    b = pyro.param("scale", torch.tensor(1.), constraint=constraints.positive)
    temp = pyro.sample('temp', dist.Normal(a, b))
```

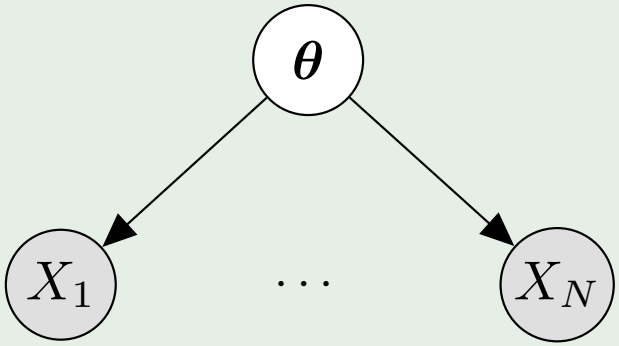
Guide requirements

1. the guide has the same input signature as the model
2. all unobserved sample statements that appear in the model appear in the guide.

Example: Pyro for a Beta-Bernoulli model

Example model

We consider again our simple generative model for thumb tack tosses:



Unfolded notation

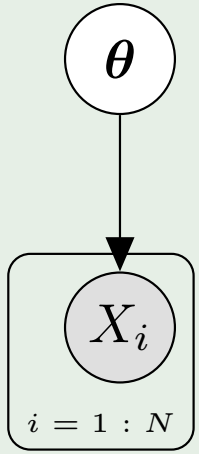


Plate notation

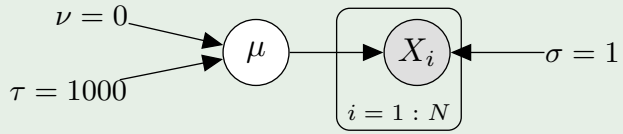
$$\theta \sim \text{Beta}(2, 5)$$

$$\theta \sim p(X_i = 1 \mid \theta) = \theta$$

thumb_tack.ipynb

Exercise

We assume a simple generative model:



Data is generated using $\mu = 10$, we assume a variational model with $q(\mu \mid \lambda) = \mathcal{N}(\lambda, 1)$, and want to maximize the ELBO wrt. the posterior mean for μ in the variational formulation.

mean_inference.ipynb

Conclusions

Variational Inference/Bayes

Variational inference: VI is a deterministic alternative to sampling for **approximate inference in Bayesian models**.

- VI seeks the model $q_\lambda(\mathbf{z})$ inside a family of applicable models $\mathcal{Q}$ that is closest to the (unattainable) posterior $p(\mathbf{z} \mid \mathbf{x})$ in terms of a Kullback Leibler divergence.
- Depending on $\mathcal{Q}$, we can assert efficient inference – a very common choice is the **mean field assumption**:

$$q_\lambda(\mathbf{z}) = \prod_i q_{\lambda_i}(z_i).$$

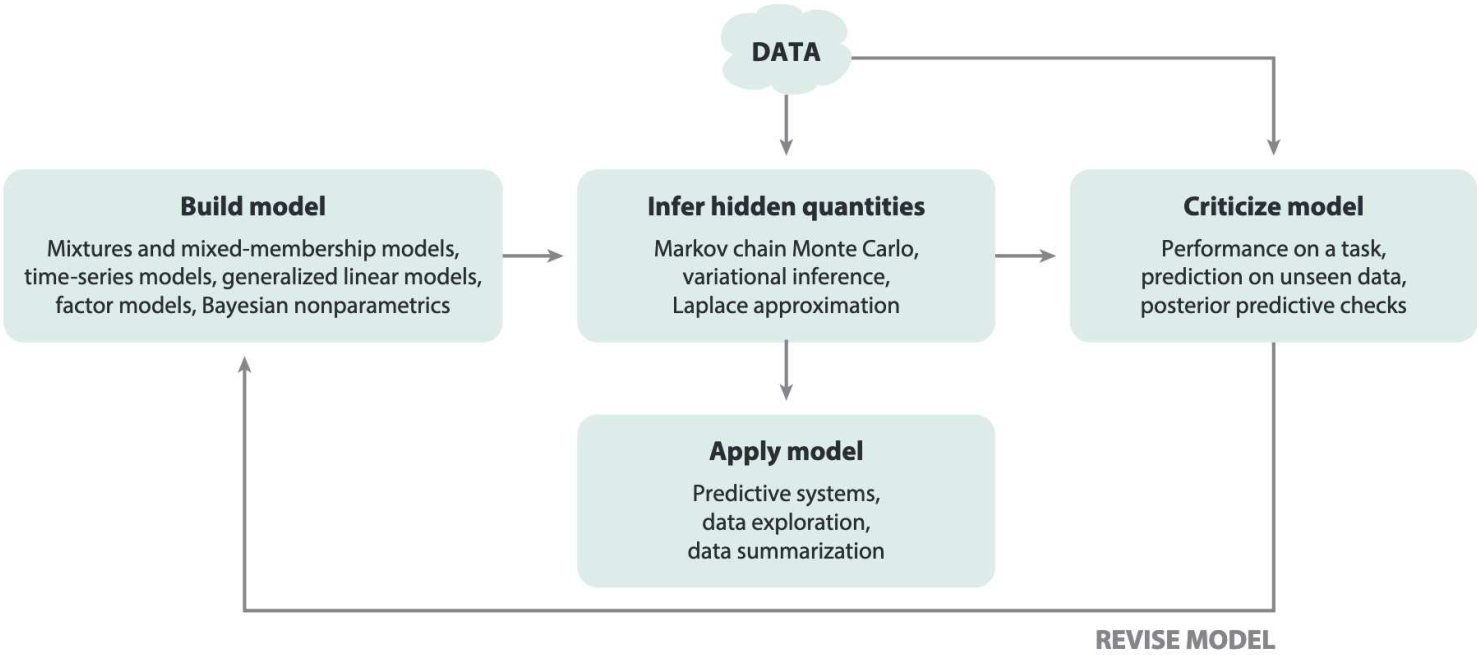
Black box variational inference:

- The goal is to obtain **efficient** inference in **unconstrained** Bayesian network models – any structure, any combination of distributional families.
- **Black-Box** Variational Inference promises VB inference in any model, but at the cost that inference relies on sampling – and it sometimes shows poor converge properties in practice.

Probabilistic Programming Languages: PPLs are programming languages to describe probabilistic models and perform inference in them.

- `Pyro` is a PPL built on top of `PyTorch`, and which supports several inference techniques, including BBVI, SVI, MCMC.
- Several other alternatives exist as well (`Edward`, `Stan`, `JAGS`, `InferPy` ...)

Box's loop



Build, Compute, Critique, Repeat: Data Analysis with Latent Variable Models by David M. Blei